

Programación 1 — Unidad 3: Estructuras repetitivas

Actividad 3: Bucles anidados, validación de entrada y eficiencia

Índice

- 3.4. Bucles anidados
- 3.6. Validar la entrada con while
- 3.7. El bloque else en los bucles
- 3.8. ¿Cuánto cuesta un algoritmo? Una primera idea de eficiencia

3.4. Bucles anidados

Un bucle puede contener otro adentro. Por cada vuelta del bucle de afuera, el de adentro se ejecuta completo. Se usan, por ejemplo, para recorrer tablas (filas y columnas) o generar combinaciones.

```
for fila in range(1, 4):
    for columna in range(1, 4):
        print(f"({fila},{columna})", end=" ")
    print()
```

Qué hace el código:

- El bucle externo `for fila...` da 3 vueltas (filas 1, 2 y 3).
- Por **cada** fila, el bucle interno `for columna...` da sus 3 vueltas completas (columnas 1, 2 y 3).
- `print(..., end=" ")` muestra cada par sin saltar de línea, separándolos con un espacio.
- El `print()` vacío del final (alineado con el `for` interno) hace el salto de línea al terminar cada fila.

3.6. Validar la entrada con while

Un uso muy común del `while` es **insistir** hasta que la persona ingrese un dato correcto. El bucle se repite mientras la entrada no sea válida, y recién continúa cuando el dato cumple lo esperado.

```
edad = -1
while edad < 0 or edad > 120:
```

```
edad = int(input("Ingresá una edad válida (0-120): "))
print(f"Edad registrada: {edad}")
```

Qué hace el código:

- `edad = -1` arranca con un valor inválido a propósito, para garantizar que el bucle se ejecute al menos una vez.
- `while edad < 0 or edad > 120:` se repite mientras la edad esté fuera del rango permitido.
- Dentro pide una nueva edad. Si la persona ingresa un valor válido, la condición se vuelve falsa y el bucle termina.
- Recién entonces se ejecuta el `print` final con la edad ya validada.

3.7. El bloque `else` en los bucles

Tanto `for` como `while` admiten un bloque `else` opcional, que se ejecuta **solo si el bucle terminó normalmente**, sin haber sido cortado por un `break`. Es muy útil en búsquedas, para distinguir «lo encontré» de «recorrí todo y no estaba».

```
clave = 7
for n in [3, 5, 8, 12]:
    if n == clave:
        print("Encontrado")
        break
else:
    print("No se encontró la clave")
```

Qué hace el código:

- El bucle recorre la lista buscando el valor de `clave` (que es 7).
- Si lo encontrara, mostraría `Encontrado` y cortaría con `break` (en ese caso el `else` NO se ejecutaría).
- Como 7 no está en la lista, el bucle termina sin `break` y entonces sí se ejecuta el `else`.
- Por eso muestra `No se encontró la clave`. El `else` va alineado con el `for`, no con el `if`.

Buenas prácticas

Elegí el bucle según el problema: usá `for` cuando sabés cuántas veces o sobre qué colección vas a iterar, y `while` cuando la repetición depende de una condición que puede cambiar de forma impredecible (por ejemplo, hasta que la persona escriba «salir»).

Encontrá el error

Este programa no termina nunca. Antes de seguir, fijate qué debería pasar en cada vuelta para que la condición del `while` deje de cumplirse.

```
vidas = 3
while vidas > 0:
    print("Te quedan", vidas, "vidas")
```

Qué estaba mal

Falta actualizar vidas dentro del bucle (por ejemplo, `vidas -= 1`). Sin eso, la condición `vidas > 0` siempre es verdadera y el bucle es infinito.

Practicá la lectura

Este ejemplo usa el bloque `else` de un bucle (sección 3.7). Sin ejecutar, escribí en orden todo lo que se imprime para `n` entre 2 y 5.

```
for n in range(2, 6):
    for d in range(2, n):
        if n % d == 0:
            print(n, "no es primo")
            break
    else:
        print(n, "es primo")
```

Para reflexionar y autoevaluarte

- ¿En qué ejercicio te costó más decidir si convenía usar `while` o `for`?
- ¿Se te ocurre algún patrón (acumular, contar, buscar) que hayas usado sin darte cuenta de que tenía nombre?

Antes de seguir, verificá que podés:

- Elegir entre `while` y `for` según el problema.
- Usar `break` y `continue` para modificar el flujo de un bucle.
- Reconocer y aplicar los patrones de acumular, contar y buscar.
- Validar una entrada de usuario con un `while` hasta obtener un valor correcto.

3.8. ¿Cuánto cuesta un algoritmo? Una primera idea de eficiencia

Dos programas pueden resolver exactamente el mismo problema y dar el mismo resultado, pero hacer una cantidad de trabajo muy distinta. Comparemos dos formas de sumar los primeros `N` números enteros.

```
total = 0
for i in range(1, N + 1):
    total += i
total = N * (N + 1) // 2
```

La primera versión hace **N sumas**: una por cada vuelta del bucle. Si `N` es 10, son 10 sumas; si `N` es 10 millones, son 10 millones de sumas.

La segunda versión, con la fórmula de Gauss, hace **siempre una sola cuenta**, sin importar cuánto valga `N`.

Analogía

Es como buscar un nombre en una guía telefónica de papel: si no sabés que está ordenada

alfabéticamente, tenés que revisar página por página (más nombres, más tiempo). Si sabés que está ordenada, podés saltar directo a la letra que buscás, casi sin importar cuán gruesa sea la guía.

Esta idea —cuánto trabajo hace un programa a medida que crece el tamaño del problema— es la base de lo que en programación se llama **eficiencia o complejidad algorítmica**.

Todavía no hace falta ponerle un nombre formal (eso se estudia con más detalle en materias como Algoritmos y Estructuras de Datos); alcanza con la idea de que **dos soluciones correctas pueden tener costos muy distintos**, y que vale la pena preguntarse cuál conviene en cada caso.